```
{ nil => nil, 1 => 1, :x => :a }.transform_values { |value|
value.to_s.upcase }
# => {nil=>"", 1=>"1", :x=>"A"}
```

There's also the bang variant `transform_values!` that applies the block operations to values in the very receiver.

Defined in `active_support/core_text/hash/transform_values.rb`.

11.6 Slicing

Ruby has built-in support for taking slices out of strings and arrays. Active Support extends slicing to hashes:

```
{a: 1, b: 2, c: 3}.slice(:a, :c)
# => {:c=>3, :a=>1}

{a: 1, b: 2, c: 3}.slice(:b, :X)
# => {:b=>2} # non-existing keys are ignored
```

If the receiver responds to `convert_key` keys are normalized:

```
{a: 1, b: 2}.with_indifferent_access.slice("a")
# => {:a=>1}
```

Slicing may come in handy for sanitizing option hashes with a white list of keys.

There's also `slice!` which in addition to perform a slice in place returns what's removed:

```
hash = {a: 1, b: 2}
rest = hash.slice!(:a) # => {:b=>2}
hash                   # => {:a=>1}
```

Defined in `active_support/core_ext/hash/slice.rb`.

11.7 Extracting

The method `extract!` removes and returns the key/value pairs matching the given keys.

```
hash = {a: 1, b: 2}
rest = hash.extract!(:a) # => {:a=>1}
hash                     # => {:b=>2}
```

The method `extract!` returns the same subclass of Hash, that the receiver is.

```
hash = {a: 1, b: 2}.with_indifferent_access
rest = hash.extract!(:a).class
# => ActiveSupport::HashWithIndifferentAccess
```

Defined in `active_support/core_ext/hash/slice.rb`.

11.8 Indifferent Access

The method `with_indifferent_access` returns an `ActiveSupport::HashWithIndifferentAccess` out of its receiver:

```
{a: 1}.with_indifferent_access["a"] # => 1
```

Defined in `active_support/core_ext/hash/indifferent_access.rb`.

11.9 Compacting

The methods `compact` and `compact!` return a Hash without items with `nil` value.

```
{a: 1, b: 2, c: nil}.compact # => {a: 1, b: 2}
```

Defined in `active_support/core_ext/hash/compact.rb`.

12 Extensions to Regexp

12.1 multiline?